

FIRE SCOUT DRONE

Software Development Roadmap

From Zero to First Real Flight

Duration 10-12 Weeks	OS Ubuntu 22.04 LTS	Simulator ArduPilot SITL + Gazebo	AI/CV YOLO v8 + OpenCV	Framework ROS2 Humble
--------------------------------	-------------------------------	--	----------------------------------	---------------------------------

This document is a **complete step-by-step roadmap** for developing the software of an autonomous fire reconnaissance drone — *without any physical hardware*. You will use simulators that behave identically to real hardware. Every step includes **exact terminal commands**, **packages to install**, **code to run**, and **success criteria**. After completing all phases, you will have production-ready software deployable on a real drone.

Phase	Title	Weeks	Key Deliverable
0	Environment Setup	1-2	Ubuntu + ROS2 + SITL running
1	Basic Navigation (SITL)	3-4	Autonomous waypoint flight
2	Building Orbit + GUIDED	5-6	Orbit algorithm + Python MAVLink API
3	Computer Vision	7-8	YOLO detects fire on video
4	CV + Drone Integration	9	Drone responds to detected fire
5	Video Stream + REST API	10	RTSP stream + REST API + Telegram
6	Final Integration	11-12	Full pipeline ready for hardware

0

PHASE 0: Development Environment Setup

Ubuntu 22.04 | ROS2 Humble | ArduPilot SITL | QGroundControl | Gazebo

Weeks 1-2

This is the foundation of the entire project. Without a correctly configured environment you cannot develop or test any code. Take your time here — each step matters.

1

Install Ubuntu 22.04 LTS

Why Ubuntu 22.04? ROS2 Humble, ArduPilot SITL, and the NVIDIA Jetson SDK all officially support exactly this version. Using a different version causes package incompatibilities.

Option A — Native machine (recommended):

- Download ISO from ubuntu.com/download/desktop — Ubuntu 22.04.4 LTS
- Write to USB (min 8 GB) using Rufus (Windows) or Balena Etcher (any OS)
- During install choose: "Minimal installation" + "Install third-party software"
- Disk partition: allocate at least 80 GB (ROS2 + models + simulator = ~30 GB)

Option B — VirtualBox / VMware (if no Linux machine):

- RAM: allocate at least 8 GB (16 GB recommended for Gazebo)
- CPU: 4+ cores, enable VT-x / AMD-V in BIOS
- Video: 3D Acceleration ON, VRAM 128 MB
- Option C — WSL2 on Windows 11: works for phases 0-3, but Gazebo GUI may not render

First system update after installation:

```
# Update package lists and upgrade the system
sudo apt update && sudo apt upgrade -y
# Install essential developer tools
sudo apt install -y git curl wget python3-pip python3-venv \
    build-essential cmake nano htop net-tools \
    software-properties-common apt-transport-https \
    ca-certificates gnupg lsb-release
```

2

Install ROS2 Humble Hawksbill

What is ROS2? Robot Operating System 2 is NOT an OS — it is a **robotics middleware framework**. It lets separate programs (navigation, camera, detector) communicate through *topics*. Humble has support until 2027.

```
# Step 1: Add ROS2 GPG key
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg
# Step 2: Add ROS2 repository
echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    http://packages.ros.org/ros2/ubuntu \
    $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
    | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
# Step 3: Install ROS2 Desktop (full version with GUI tools)
sudo apt update
```

```

sudo apt install -y ros-humble-desktop
# Step 4: Install build tools and dependency manager
sudo apt install -y python3-colcon-common-extensions \
    python3-rosdep python3-argcomplete
# Step 5: Initialize rosdep (ROS dependency manager)
sudo rosdep init
rosdep update
# Step 6: Auto-load ROS2 in every new terminal
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
# Step 7: Verify installation
ros2 --version
# Expected output: ros2 cli version X.X.X

```

IMPORTANT: After installation, close and reopen the terminal. The "ros2" command must work without any extra steps.

3 Install ArduPilot SITL (drone simulator)

What is SITL? Software In The Loop — ArduPilot (the same code that runs on Pixhawk) compiled to run on your PC. It simulates all sensors (GPS, IMU, barometer), responds to MAVLink commands, and behaves *identically* to real hardware.

```

# Step 1: Clone ArduPilot repository
cd ~
git clone https://github.com/ArduPilot/ardupilot.git
cd ardupilot
# Step 2: Initialize submodules (required!)
git submodule update --init --recursive
# Step 3: Run automatic dependency installer
# This script installs ~50 packages. Takes 5-10 minutes.
Tools/environment_install/install-prereqs-ubuntu.sh -y
# Step 4: Apply environment changes
. ~/.profile
# Step 5: Build SITL for ArduCopter (multirotor)
# First build takes 10-15 minutes
./waf configure --board sitl
./waf copter
# Step 6: Test launch of SITL
cd ArduCopter
sim_vehicle.py -v ArduCopter --console --map
# Expected: console with telemetry + map window with drone marker

```

SUCCESS: Console shows "APM: ArduCopter ... Ready to Fly" and a drone marker appears on the map.

4 Install MAVROS (ROS2 <-> ArduPilot bridge)

What is MAVROS? A ROS2 package that translates MAVLink (ArduPilot protocol) into ROS2 topics. Thanks to MAVROS, your Python code sends drone commands via the standard ROS2 API instead of raw MAVLink bytes.

```

# Install MAVROS and geographic data libraries
sudo apt install -y ros-humble-mavros ros-humble-mavros-extras
# MANDATORY: install geographic datasets for MAVROS
sudo /opt/ros/humble/lib/mavros/install_geographiclib_datasets.sh
# Install Python MAVLink libraries
pip3 install pymavlink mavproxy dronekit --break-system-packages

```

```
# --- TEST: connect MAVROS to SITL ---
# Terminal 1: start SITL
sim_vehicle.py -v ArduCopter --console
# Terminal 2: start MAVROS
ros2 launch mavros apm.launch fcu_url:=udp://127.0.0.1:14550@
# Terminal 3: check drone state
ros2 topic echo /mavros/state
# Expected: connected: true, armed: false, mode: "STABILIZE"
```

5 Install QGroundControl (Ground Control Station)

QGroundControl is a free GCS for mission planning, telemetry monitoring, and ArduPilot parameter configuration. Recommended over Mission Planner on Linux.

```
# Download QGroundControl AppImage
cd ~/Downloads
wget https://d176tv9ibo4jno.cloudfront.net/latest/QGroundControl.AppImage
chmod +x QGroundControl.AppImage
# Launch
./QGroundControl.AppImage
# QGC auto-connects to SITL on UDP 14550
# You will see the drone on the map with live telemetry
```

6 Install Gazebo Garden (3D Physics Simulator)

Why Gazebo? SITL simulates flight math but has no 3D visualization. Gazebo adds a realistic 3D scene with buildings, wind, and physics — so you can see the drone orbiting a building.

```
# Add Gazebo Garden repository
sudo curl -sSL https://packages.osrfoundation.org/gazebo.gpg \
-o /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] \
http://packages.osrfoundation.org/gazebo/ubuntu-stable \
$(lsb_release -cs) main" \
| sudo tee /etc/apt/sources.list.d/gazebo-stable.list
# Install Gazebo Garden
sudo apt update
sudo apt install -y gz-garden
# Build and install ArduPilot-Gazebo plugin (connects SITL + Gazebo)
sudo apt install -y libgz-sim7-dev rapidjson-dev
cd ~
git clone https://github.com/ArduPilot/ardupilot_gazebo.git
cd ardupilot_gazebo && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo
make -j$(nproc)
sudo make install
# Add Gazebo paths to .bashrc
echo "export GZ_SIM_SYSTEM_PLUGIN_PATH=/usr/local/lib/gz/sim7/plugins" >> ~/.bashrc
echo "export GZ_SIM_RESOURCE_PATH=~/.ardupilot_gazebo/models:~/.ardupilot_gazebo/worlds" >> ~/.bashrc
source ~/.bashrc
# Test Gazebo
gz sim
# Expected: 3D Gazebo window opens
```

7 Create Project Structure (ROS2 Workspace)

```
# Create ROS2 workspace
mkdir -p ~/fire_scout_ws/src
cd ~/fire_scout_ws
# Create navigation package
cd src
ros2 pkg create fire_scout_nav --build-type ament_python \
  --dependencies rclpy mavros_msgs geometry_msgs sensor_msgs
# Create computer vision package
ros2 pkg create fire_scout_cv --build-type ament_python \
  --dependencies rclpy sensor_msgs cv_bridge
# Create API / alerts package
ros2 pkg create fire_scout_api --build-type ament_python \
  --dependencies rclpy std_msgs
# Build workspace
cd ~/fire_scout_ws
colcon build
source install/setup.bash
# Add to .bashrc so it loads automatically
echo "source ~/fire_scout_ws/install/setup.bash" >> ~/.bashrc
```

Phase 0 — Success Criteria:

- OK "ros2 --version" returns a version without errors
- OK SITL starts: sim_vehicle.py shows "Ready to Fly" and a map with drone
- OK "ros2 topic echo /mavros/state" shows connected: true
- OK QGroundControl connects to SITL and shows live telemetry
- OK "gz sim" opens the 3D Gazebo window
- OK Workspace ~/fire_scout_ws builds without errors (colcon build)

1

PHASE 1: Basic Autonomous Navigation

Weeks 3-4

First autonomous flight | Waypoints | Auto takeoff/land | RTL failsafe

In this phase you will write the first Python script that makes the simulated drone take off, fly to given coordinates, and return home. This is the foundation for all subsequent mission logic.

1

Install Python navigation dependencies

```
# DroneKit: high-level Python API for ArduPilot
pip3 install dronekit --break-system-packages
# Geopy: GPS distance calculations
pip3 install geopy --break-system-packages
# Verify
python3 -c "import dronekit; print('dronekit OK!')"
python3 -c "import pymavlink; print('pymavlink OK!')"
```

2

First test: connect to SITL and read telemetry

Before writing a full mission script — verify that you can connect and read telemetry. File: `~/fire_scout_ws/src/fire_scout_nav/fire_scout_nav/test_connect.py`

```
# test_connect.py
from dronekit import connect, VehicleMode
import time
# Connect to SITL (SITL listens on UDP port 14550)
print("Connecting to SITL...")
vehicle = connect("udp:127.0.0.1:14550", wait_ready=True)
# Print telemetry
print(f"ArduPilot version : {vehicle.version}")
print(f"Flight mode      : {vehicle.mode.name}")
print(f"GPS position       : {vehicle.location.global_frame}")
print(f"Altitude (rel)     : {vehicle.location.global_relative_frame.alt} m")
print(f"Battery voltage    : {vehicle.battery.voltage} V")
print(f"GPS fix type       : {vehicle.gps_0.fix_type}")
# Listen for mode changes
@vehicle.on_attribute("mode")
def mode_callback(self, attr_name, value):
    print(f"Mode changed to: {value.name}")
time.sleep(10)
vehicle.close()
print("Disconnected")
# Run: python3 test_connect.py (SITL must be running in another terminal)
```

3

Mission Manager — takeoff and land

This is the core flight control class. The `arm_and_takeoff()` function is the most critical piece of code and will be reused in every mission.

File: `fire_scout_nav/mission_manager.py`

```
"""
mission_manager.py - core flight control for Fire Scout
```

```

Works identically with SITL and real Pixhawk (only connection string differs)
"""
from dronekit import connect, VehicleMode, LocationGlobalRelative
import time, math
class MissionManager:
    def __init__(self, connection="udp:127.0.0.1:14550"):
        print(f"[INIT] Connecting to: {connection}")
        self.vehicle = connect(connection, wait_ready=True)
        print("[INIT] Connected successfully")
    def arm_and_takeoff(self, target_altitude):
        """Arm motors and take off to target altitude (metres)."""
        print(f"[TAKEOFF] Target altitude: {target_altitude} m")
        # Wait for GPS 3D fix (fix_type >= 2)
        print("[TAKEOFF] Waiting for GPS fix...")
        while self.vehicle.gps_0.fix_type < 2:
            print(f"GPS fix type: {self.vehicle.gps_0.fix_type}")
            time.sleep(1)
        # Switch to GUIDED mode (required before arming)
        self.vehicle.mode = VehicleMode("GUIDED")
        while self.vehicle.mode.name != "GUIDED":
            time.sleep(0.5)
        print("[TAKEOFF] Mode: GUIDED")
        # Arm motors
        self.vehicle.armed = True
        while not self.vehicle.armed:
            print("Waiting for ARM...")
            time.sleep(1)
        print("[TAKEOFF] Motors ARMED")
        # Send takeoff command
        self.vehicle.simple_takeoff(target_altitude)
        # Wait until target altitude reached
        while True:
            alt = self.vehicle.location.global_relative_frame.alt
            print(f"Altitude: {alt:.1f} m (target: {target_altitude} m)")
            if alt >= target_altitude * 0.95:
                print("[TAKEOFF] Target altitude reached")
                break
            time.sleep(1)
    def goto(self, lat, lon, alt):
        """Fly to GPS coordinates at given altitude."""
        target = LocationGlobalRelative(lat, lon, alt)
        self.vehicle.simple_goto(target)
        print(f"[GOTO] Flying to: {lat:.6f}, {lon:.6f}, {alt} m")
    def return_to_launch(self):
        """Return to home position (RTL)."""
        print("[RTL] Returning home...")
        self.vehicle.mode = VehicleMode("RTL")
    def land(self):
        """Land at current position."""
        print("[LAND] Landing...")
        self.vehicle.mode = VehicleMode("LAND")
    def close(self):
        self.vehicle.close()
# Quick test: takeoff -> hover 10s -> RTL
if __name__ == "__main__":

```

```
mgr = MissionManager()
mgr.arm_and_takeoff(20)
print("Hovering for 10 seconds...")
time.sleep(10)
mgr.return_to_launch()
time.sleep(20)
mgr.close()
```

4 Waypoint mission — autonomous route

Fly through several GPS waypoints automatically. The default SITL start location is KSFO airport (San Francisco). We use relative offsets so the test works at any location.

```
# waypoint_mission.py
from dronekit import connect, VehicleMode, LocationGlobalRelative
import time, math

def distance_metres(loc1, loc2):
    """Approximate distance in metres between two GPS points."""
    dlat = loc2.lat - loc1.lat
    dlon = loc2.lon - loc1.lon
    return math.sqrt((dlat * 1.113195e5)**2 + (dlon * 1.113195e5)**2)

def offset_location(lat, lon, d_north, d_east):
    """Offset GPS coords by N/E metres – useful for relative tests."""
    R = 6378137.0
    dlat = d_north / R
    dlon = d_east / (R * math.cos(math.radians(lat)))
    return lat + math.degrees(dlat), lon + math.degrees(dlon)

vehicle = connect("udp:127.0.0.1:14550", wait_ready=True)
home_lat = vehicle.location.global_frame.lat
home_lon = vehicle.location.global_frame.lon

# Generate 4 waypoints relative to home
waypoints = [
    offset_location(home_lat, home_lon, 50, 0), # 50 m North
    offset_location(home_lat, home_lon, 50, 50), # NE diagonal
    offset_location(home_lat, home_lon, 0, 50), # 50 m East
    offset_location(home_lat, home_lon, 0, 0), # back to home
]

# Arm and take off
vehicle.mode = VehicleMode("GUIDED")
vehicle.armed = True
while not vehicle.armed: time.sleep(0.5)
vehicle.simple_takeoff(30)
while vehicle.location.global_relative_frame.alt < 28: time.sleep(1)

# Fly waypoints
for i, (lat, lon) in enumerate(waypoints):
    target = LocationGlobalRelative(lat, lon, 30)
    vehicle.simple_goto(target)
    print(f"[WP {i+1}] Flying to {lat:.5f}, {lon:.5f}")
    while True:
        d = distance_metres(vehicle.location.global_frame, target)
        print(f" Distance to waypoint: {d:.1f} m")
        if d < 3: break
        time.sleep(1)
    print(f"[WP {i+1}] Reached")
vehicle.mode = VehicleMode("LAND")
print("[MISSION] Complete")
```



```
vehicle.close()
```

Phase 1 — Success Criteria:

OK test_connect.py prints ArduPilot version and GPS coordinates without errors

OK mission_manager.py: drone takes off to 20 m visible in QGC, then RTL

OK waypoint_mission.py: drone visits all 4 waypoints in correct order

OK Deviation from each waypoint is less than 3 m (visible on QGC map)

OK RTL triggers correctly when RC signal is cut in SITL console

2

PHASE 2: Building Orbit & GUIDED Mode

Orbit algorithm | Dynamic waypoint injection | ROS2 navigation node

Weeks 5-6

The building orbit is the key fire reconnaissance maneuver: fly a circle around a building at a given radius and altitude, hovering when fire is detected.

1

Building orbit algorithm

```
# building_orbit.py
import math
from dronekit import connect, VehicleMode, LocationGlobalRelative
import time

def generate_orbit(center_lat, center_lon, radius_m, altitude_m, num_points=12):
    """
    Generate a list of GPS waypoints in a circle around a building.
    center_lat/lon : GPS coordinates of building centre
    radius_m       : orbit radius in metres (building_width/2 + 10 m recommended)
    altitude_m     : flight altitude (building_height + 10 m recommended)
    num_points     : number of waypoints (more = smoother circle)
    """
    EARTH_RADIUS = 6371000.0
    waypoints = []
    for i in range(num_points):
        angle = (2 * math.pi * i) / num_points
        d_lat = (radius_m * math.cos(angle)) / EARTH_RADIUS
        d_lon = (radius_m * math.sin(angle)) / \
            (EARTH_RADIUS * math.cos(math.radians(center_lat)))
        waypoints.append(LocationGlobalRelative(
            center_lat + math.degrees(d_lat),
            center_lon + math.degrees(d_lon),
            altitude_m
        ))
    return waypoints

def orbit_building(vehicle, center_lat, center_lon,
                  radius_m=25, altitude_m=30, speed_ms=3):
    """Execute a full orbit around the building."""
    print(f"[ORBIT] Centre: {center_lat:.6f}, {center_lon:.6f}")
    print(f"[ORBIT] Radius: {radius_m} m Altitude: {altitude_m} m")
    vehicle.airspeed = speed_ms
    wps = generate_orbit(center_lat, center_lon, radius_m, altitude_m)
    for i, wp in enumerate(wps):
        vehicle.simple_goto(wp)
        print(f"[ORBIT] Point {i+1}/{len(wps)}")
        while True:
            curr = vehicle.location.global_frame
            dlat = wp.lat - curr.lat
            dlon = wp.lon - curr.lon
            dist = math.sqrt((dlat*1.113195e5)**2 + (dlon*1.113195e5)**2)
            if dist < 2.0: break
            time.sleep(0.5)
    print("[ORBIT] Orbit complete")
```

2 ROS2 Navigation Node (production-grade)

Rewrite the navigation logic as a proper ROS2 node. This allows other nodes (CV detector, API server) to send commands via topics using JSON.

```
# ros2_nav_node.py
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from mavros_msgs.msg import State
from mavros_msgs.srv import CommandBool, SetMode
import json

class FireScoutNav(Node):
    def __init__(self):
        super().__init__("fire_scout_nav")
        self.state_sub = self.create_subscription(
            State, "/mavros/state", self.state_callback, 10)
        self.cmd_sub = self.create_subscription(
            String, "/fire_scout/nav_command", self.command_callback, 10)
        self.arming_client = self.create_client(CommandBool, "/mavros/cmd/arming")
        self.set_mode_client = self.create_client(SetMode, "/mavros/set_mode")
        self.current_state = State()
        self.get_logger().info("FireScoutNav node started")
    def state_callback(self, msg):
        self.current_state = msg
    def command_callback(self, msg):
        """Handle JSON commands from CV node or API server."""
        try:
            cmd = json.loads(msg.data)
            action = cmd.get("action")
            if action == "takeoff":
                self.do_takeoff(cmd.get("altitude", 20))
            elif action == "goto":
                self.do_goto(cmd["lat"], cmd["lon"], cmd.get("alt", 30))
            elif action == "orbit":
                self.do_orbit(cmd["lat"], cmd["lon"],
                             cmd.get("radius", 25), cmd.get("alt", 30))
            elif action == "rtl":
                self.do_rtl()
            elif action == "loiter":
                self.do_loiter()
        except Exception as e:
            self.get_logger().error(f"Command error: {e}")

def main():
    rclpy.init()
    rclpy.spin(FireScoutNav())
    rclpy.shutdown()
```

Send a test orbit command from terminal:

```
ros2 topic pub --once /fire_scout/nav_command std_msgs/msg/String \
 '{"data": "{\\"action\\": \\"orbit\\", \\"lat\\": 37.6236, \\"lon\\": -122.0822, \\"radius\\": 30, \\"alt\\": 25}\\"}'
```

Phase 2 — Success Criteria:

OK building_orbit.py: drone orbits a centre point at 25 m radius (visible in QGC)

OK Circle closes — last waypoint connects back to first

OK ROS2 nav node receives JSON "orbit" command and executes it

OK JSON "rtl" command immediately triggers return-to-launch

OK Orbit deviation stays within 3 m throughout the full circle

3

PHASE 3: Computer Vision — Fire Detection

YOLO v8 | Dataset download | Model training | ROS2 CV node

Weeks 7-8

This phase is completely independent of the drone — you train and validate a fire/smoke/person detector. You only need a laptop or PC.

1 Install YOLO v8 and CV dependencies

```
# Install Ultralytics YOLO v8
pip3 install ultralytics --break-system-packages
# OpenCV with full support
pip3 install opencv-python-headless --break-system-packages
# Data processing and augmentation
pip3 install matplotlib pillow albumentations --break-system-packages
# Roboflow — for downloading annotated datasets
pip3 install roboflow --break-system-packages
# Check GPU availability (optional but speeds up training 10x)
python3 -c "import torch; print('GPU available:', torch.cuda.is_available())"
# True = NVIDIA GPU ready   False = CPU will be used (slower but works)
# Quick YOLO sanity check
python3 -c "from ultralytics import YOLO; YOLO('yolov8n.pt'); print('YOLO v8 OK')"
```

2 Download fire detection dataset

Where to get training data? Roboflow Universe is a free platform with thousands of annotated datasets. You need classes: fire and smoke.

```
# Option 1: Download via Roboflow Python API
# (requires free account at roboflow.com)
from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_FREE_API_KEY")
project = rf.workspace("firesmokedetect").project("fire-smoke-detection-5lp0a")
dataset = project.version(4).download("yolov8")
# Option 2: Manual — Roboflow Universe -> search "fire smoke"
# -> Export Format: YOLOv8 -> Download ZIP -> unzip to ~/fire_dataset/
# Resulting directory structure:
# fire_dataset/
#   data.yaml      <- dataset config (class names, paths)
#   train/ images/ labels/
#   valid/ images/ labels/
#   test/  images/ labels/
# Verify dataset
python3 -c "
import yaml
with open('fire_dataset/data.yaml') as f: d=yaml.safe_load(f)
print('Classes:', d['names'])
print('Train images:', len(list(__import__('pathlib').Path('fire_dataset/train/images').glob('*.jpg')
)))
"
```

3 Train YOLO v8 model

```
# train_fire_model.py
from ultralytics import YOLO
# Load base model: yolov8n = nano (fastest, best for drone)
# n=nano, s=small, m=medium – nano is enough for real-time on Jetson
model = YOLO("yolov8n.pt")
# Train
results = model.train(
    data      = "fire_dataset/data.yaml",
    epochs    = 50,          # 50 epochs is a good starting point
    imgsz     = 640,        # input image size
    batch     = 16,         # reduce to 8 if RAM is limited
    name      = "fire_scout_v1",
    patience  = 10,         # stop if no improvement for 10 epochs
    device    = "0",        # "0" = first GPU, "cpu" = processor
    augment   = True,       # data augmentation improves accuracy
)
# Trained model saved at:
# runs/detect/fire_scout_v1/weights/best.pt
metrics = results.results_dict
print(f"mAP50: {metrics['metrics/mAP50(B)']:.3f}")
# mAP50 > 0.70 is a good result for fire detection
# Quick test on a video file (no camera needed)
model_trained = YOLO("runs/detect/fire_scout_v1/weights/best.pt")
model_trained("test_fire_video.mp4", save=True, conf=0.6)
# Output video saved to runs/detect/predict/
```

Training time: CPU-only ~2-4 hours. NVIDIA GTX 1060+ ~15-30 min. No GPU? Use free Google Colab (Runtime -> Change runtime -> T4 GPU).

4

Fire Detector ROS2 Node

```
# fire_detector_node.py
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
from ultralytics import YOLO
import json

class FireDetectorNode(Node):
    def __init__(self):
        super().__init__("fire_detector")
        model_path = self.declare_parameter(
            "model_path", "models/fire_scout_v1.pt").value
        self.model = YOLO(model_path)
        self.bridge = CvBridge()
        # Subscribe to camera image topic
        self.img_sub = self.create_subscription(
            Image, "/camera/image_raw", self.image_callback, 1)
        # Publish fire alerts
        self.alert_pub = self.create_publisher(
            String, "/fire_scout/fire_alert", 10)
        # Publish annotated video for monitoring
        self.viz_pub = self.create_publisher(
            Image, "/fire_scout/detection_viz", 1)
```

```

        self.get_logger().info(f"FireDetector ready: {model_path}")
def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
    results = self.model(frame, conf=0.6, verbose=False)
    detections = []
    for r in results:
        for box in r.bboxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            label = self.model.names[cls_id]
            if label in ["fire", "smoke"]:
                x1,y1,x2,y2 = map(int, box.xyxy[0])
                detections.append({
                    "label": label, "confidence": round(conf,3),
                    "bbox": [x1,y1,x2,y2]
                })
    if detections:
        alert = json.dumps({
            "type": "FIRE_DETECTED",
            "detections": detections,
            "ts": self.get_clock().now().to_msg().sec
        })
        self.alert_pub.publish(String(data=alert))
        self.get_logger().warn(f"FIRE! {len(detections)} objects")
    viz = results[0].plot()
    self.viz_pub.publish(self.bridge.cv2_to_imgmsg(viz, "bgr8"))
def main():
    rclpy.init()
    rclpy.spin(FireDetectorNode())
    rclpy.shutdown()

```

Phase 3 — Success Criteria:

- OK** Dataset downloaded: train/valid/test folders with images and labels
- OK** Training completes without errors, mAP50 > 0.70
- OK** Test on video file: YOLO visibly detects fire/smoke with bounding boxes
- OK** ROS2 node starts: `ros2 run fire_scout_cv fire_detector`
- OK** Alert JSON published to `/fire_scout/fire_alert` when fire frame is fed

4

PHASE 4: CV + Navigation Integration

Week 9

Drone responds to detected fire | Auto-hover | Mission Coordinator | Launch file

Connect phases 1-3 together: drone flies orbit, CV node detects fire, navigation node hovers and sends alert to dispatcher.

1

Mission Coordinator — the main orchestrator

```
# mission_coordinator.py
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import json

class MissionCoordinator(Node):
    def __init__(self):
        super().__init__("mission_coordinator")
        # Listen for fire alerts from CV node
        self.alert_sub = self.create_subscription(
            String, "/fire_scout/fire_alert",
            self.fire_detected_callback, 10)
        # Send commands to navigation node
        self.nav_pub = self.create_publisher(
            String, "/fire_scout/nav_command", 10)
        # Publish incident reports
        self.report_pub = self.create_publisher(
            String, "/fire_scout/incident_report", 10)
        self.fire_response_active = False
        self.get_logger().info("Mission Coordinator ready")
    def fire_detected_callback(self, msg):
        if self.fire_response_active:
            return # already responding
        data = json.loads(msg.data)
        self.get_logger().warn("FIRE DETECTED – switching to LOITER")
        # Command drone to hold position (LOITER)
        self.fire_response_active = True
        loiter_cmd = json.dumps({"action": "loiter"})
        self.nav_pub.publish(String(data=loiter_cmd))
        # Build incident report
        report = json.dumps({
            "event"      : "FIRE_DETECTED",
            "detections": data["detections"],
            "timestamp"  : data["ts"],
            "note"       : "Drone hovering at detection point"
        })
        self.report_pub.publish(String(data=report))
        self.get_logger().info("Incident report published")
def main():
    rclpy.init()
    rclpy.spin(MissionCoordinator())
    rclpy.shutdown()
```


2

ROS2 Launch File — start everything with one command

```
# fire_scout_launch.py
from launch import LaunchDescription
from launch_ros.actions import Node
def generate_launch_description():
    return LaunchDescription([
        # Navigation node
        Node(package="fire_scout_nav",
            executable="nav_node",
            name="fire_scout_nav",
            output="screen"),
        # Fire detection node
        Node(package="fire_scout_cv",
            executable="fire_detector",
            name="fire_detector",
            parameters=[{"model_path": "models/fire_scout_v1.pt"}],
            output="screen"),
        # Mission coordinator
        Node(package="fire_scout_nav",
            executable="mission_coordinator",
            name="mission_coordinator",
            output="screen"),
    ])
# Start entire system:
# ros2 launch fire_scout_nav fire_scout_launch.py
```

Phase 4 — Success Criteria:

- OK** Launch file starts all 3 nodes without errors
- OK** Drone executes orbit mission in SITL
- OK** Publishing a test alert to /fire_scout/fire_alert causes drone to switch to LOITER
- OK** Incident report published to /fire_scout/incident_report with detections
- OK** System runs continuously for 10 minutes without crash or memory leak

5

PHASE 5: Video Stream + REST API + Alerts

Week 10

FastAPI server | Telegram bot | RTSP video stream | WebSocket status

1

FastAPI REST server — control drone over HTTP

```
# Install FastAPI stack
pip3 install fastapi uvicorn python-telegram-bot --break-system-packages
# api_server.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import json

app = FastAPI(title="Fire Scout API", version="1.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                  allow_methods=["*"], allow_headers=["*"])

class MissionRequest(BaseModel):
    lat : float
    lon : float
    altitude : float = 30
    mission_type : str # "orbit" | "waypoint" | "rtl"
    radius : float = 25 # orbit radius in metres

@app.post("/api/v1/mission/start")
async def start_mission(req: MissionRequest):
    """Send a new mission with building coordinates."""
    cmd = json.dumps({
        "action": req.mission_type,
        "lat": req.lat, "lon": req.lon,
        "alt": req.altitude, "radius": req.radius
    })
    ros_node.nav_pub.publish(String(data=cmd))
    return {"status": "mission_started", "mission": req.dict()}

@app.get("/api/v1/status")
async def get_status():
    """Current drone status."""
    return {
        "mode" : ros_node.current_state.mode,
        "armed" : ros_node.current_state.armed,
        "battery_pct": ros_node.battery_pct,
        "gps" : ros_node.current_gps,
        "alerts" : ros_node.alerts_count
    }

@app.post("/api/v1/rtl")
async def rtl():
    """Immediate return to launch."""
    ros_node.nav_pub.publish(String(data=json.dumps({"action": "rtl"})))
    return {"status": "rtl_initiated"}

# Start server:
# uvicorn api_server:app --host 0.0.0.0 --port 8080
# Auto-generated docs: http://localhost:8080/docs
```

2

Telegram Bot for instant alerts

1. Register a bot via @BotFather in Telegram — receive a token. 2. Find your chat_id via @userinfobot.

```
# telegram_alert.py
import asyncio
from telegram import Bot
class TelegramAlerter:
    def __init__(self, token: str, chat_id: str):
        self.bot = Bot(token=token)
        self.chat_id = chat_id
    async def send_fire_alert(self, lat, lon, confidence, detections):
        msg = (
            f"FIRE DETECTED\n"
            f"Coordinates : {lat:.6f}, {lon:.6f}\n"
            f"Confidence   : {confidence*100:.0f}%\n"
            f"Objects        : {detections}\n"
            f"Map             : https://maps.google.com/?q={lat},{lon}"
        )
        await self.bot.send_message(chat_id=self.chat_id, text=msg)
    def alert(self, lat, lon, conf, det):
        asyncio.run(self.send_fire_alert(lat, lon, conf, det))
```

Phase 5 — Success Criteria:

OK API server running: <http://localhost:8080/docs> opens with all endpoints

OK POST /api/v1/mission/start triggers orbit in SITL

OK GET /api/v1/status returns current drone state in real time

OK POST /api/v1/rtl returns the drone home immediately

OK Telegram bot sends a message within 3 seconds of a test fire alert

6

PHASE 6: Final Integration & Hardware Handoff

Weeks 11-12

End-to-end test | Safety parameters | One-line switch from SITL to Pixhawk

1

End-to-end scenario test (UC-1)

Test scenario: Dispatcher enters building coordinates via API -> drone takes off -> orbits building -> CV detects test fire (image shown on screen) -> drone hovers -> Telegram alert sent -> RTL triggered.

```
# Terminal 1: Start SITL
cd ~/ardupilot/ArduCopter
sim_vehicle.py -v ArduCopter --console

# Terminal 2: Start MAVROS
ros2 launch mavros apm.launch fcu_url:=udp://127.0.0.1:14550@

# Terminal 3: Start Fire Scout system
cd ~/fire_scout_ws && source install/setup.bash
ros2 launch fire_scout_nav fire_scout_launch.py

# Terminal 4: Start API server
uvicorn api_server:app --host 0.0.0.0 --port 8080

# Test via curl:
curl -X POST http://localhost:8080/api/v1/mission/start \
  -H "Content-Type: application/json" \
  -d '{"lat": 37.6236, "lon": -122.0822, "altitude": 30,
      "mission_type": "orbit", "radius": 25}'

# Expected response:
# {"status": "mission_started", "mission": {...}}
```

2

ArduPilot safety parameters for real hardware

Prepare this configuration file before receiving hardware. Load in Mission Planner: Config -> Full Parameter List -> Load.

```
# safety_params.param (load into Mission Planner before first real flight)
# Battery failsafe (6S LiPo: full=25.2V, warning=22V, critical=21V)
BATT_LOW_VOLT,22.0
BATT_CRT_VOLT,21.0
BATT_FS_LOW_ACT,2    # Low battery -> RTL
BATT_FS_CRT_ACT,1    # Critical bat -> Land immediately
# Communication failsafe
FS_THR_ENABLE,1      # RC signal lost -> RTL
FS_GCS_ENABLE,1      # GCS link lost -> LOITER 30s -> RTL
# Return-to-launch settings
RTL_ALT,5000         # RTL altitude = 50 m (value is cm)
RTL_LOIT_TIME,500    # Hover 5 sec before landing at home
# Geofence (UAV regulations Ukraine: max 120 m AGL)
FENCE_ENABLE,1
FENCE_TYPE,3         # Circle + Altitude fence
FENCE_RADIUS,500     # Max 500 m from launch point
FENCE_ALT_MAX,100    # Max 100 m altitude
FENCE_ACTION,1       # Fence breach -> RTL
# Navigation speeds
WPNAV_SPEED,500      # AUTO mode speed: 5 m/s (value in cm/s)
LAND_SPEED,50        # Landing descent: 0.5 m/s (careful!)
```

